

pa7uiclf

Ciência da Computação

O TAD Árvore Binária

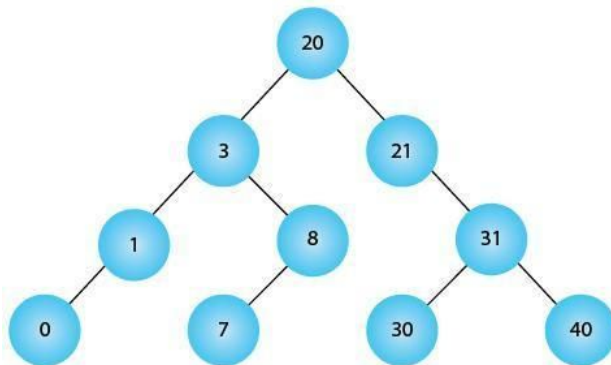
14/11/2025

O TAD Árvore Binária

- TAD Árvore Binária
- Conceitos Básicos
- Propriedades
- Operações do TAD Árvore Binária
- Representações do TAD
- Características do TAD
- Terminologia Essencial
- Implementação do Nó
- Classe Árvore Binária
- Percursos (Traversal)
- Aplicações Reais
- Algoritmos

TAD Árvore Binária

- Uma **Árvore Binária** é uma estrutura de dados hierárquica onde **cada nó pode ter no máximo dois filhos**, chamados de **filho esquerdo** e **filho direito**.
-
- O **TAD Árvore Binária** define *o que* a árvore faz (operações) e *como ela deve se comportar*, sem especificar detalhes de implementação.



Conceitos Básicos

- **Nó (Node)**

- Estrutura fundamental que contém:
 - um **valor** (dado)
 - uma referência para o **filho esquerdo**
 - uma referência para o **filho direito**

- **Raiz (Root)**

- É o nó principal, de onde a árvore começa.

- **Subárvore**

- Qualquer nó pode ser a raiz de uma nova árvore (subárvore esquerda ou direita).

- **Altura**

- Maior número de níveis da árvore (da raiz até o nó mais profundo).

- **Árvore vazia**

- Quando não há nós.

Propriedades

Cada nó tem **0, 1 ou 2** filhos.

A estrutura é **recursiva** (uma árvore é composta por subárvores).

A ordem esquerda/direita **importa**.

Árvores podem ser **completas, cheias, perfeitas** ou **desbalanceadas**.

Operações do TAD Árvore Binária

O TAD define operações abstratas como:

♦ Criar

- `criaArvoreVazia()`
- `criaArvore(valor, esq, dir)`

♦ Verificações

- `estaVazia(arvore)`
- `ehFolha(no)`
- `altura(arvore)`
- `tamanho(arvore)` (número de nós)

♦ Acesso a Partes

- `raiz(arvore)`
- `filhoEsquerdo(arvore)`
- `filhoDireito(arvore)`

♦ Modificações

- `insereEsquerda(no, valor)`
- `insereDireita(no, valor)`
- `remove(no)` (dependendo da estratégia)

♦ Percursos (travessias)

Usados para processar todos os nós:

- **Pré-ordem:** raiz → esquerda → direita
- **Em-ordem:** esquerda → raiz → direita
- **Pós-ordem:** esquerda → direita → raiz
- **Por nível:** do topo para baixo, nível por nível

Representações do TAD

Existem duas formas principais:

1) Representação Encadeada (lista de nós)

Cada nó tem ponteiros:

```
Nó {  
  
    dado  
  
    *esq  
  
    *dir  
  
}
```

2) Representação Sequencial (vetor)

Muito usada para **heaps**:

- raiz na posição 1
- filho esquerdo: $2*i$
- filho direito: $2*i + 1$

Características do TAD

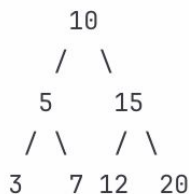
O TAD define:

- **Quais operações existem**
- **Quais garantias elas oferecem**
- **Como o usuário interage com a abstração**
- **O que cada operação faz** (não o *como*)

Ele *não* especifica:

- detalhes do armazenamento
- implementação em linguagens específicas
- otimizações internas

Terminologia Essencial



Termo	Definição	Exemplo
Raiz	Nó sem pai	10
Folha	Nó sem filhos	3, 7, 12, 20
Nó Interno	Nó com pelo menos 1 filho	5, 15
Altura	Caminho mais longo até folha	2
Profundidade	Distância até a raiz	nó 3 tem profundidade 2
Subárvore	Árvore formada por um nó e descendentes	subárvore esquerda de 10

Implementação do Nó

java



```
class No {  
    int valor;  
    No esquerdo;  
    No direito;  
  
    public No(int valor) {  
        this.valor = valor;  
        this.esquerdo = null;  
        this.direito = null;  
    }  
}
```

Classe Árvore Binária

java

```
class ArvoreBinaria {  
    private No raiz;  
  
    public ArvoreBinaria() {  
        this.raiz = null;  
    }  
  
    // Métodos virão aqui...  
}
```

Percursos (Traversal)

Usados para processar todos os nós:

- **Pré-ordem:** raiz → esquerda → direita
- **Em-ordem:** esquerda → raiz → direita
- **Pós-ordem:** esquerda → direita → raiz
- **Por nível:** do topo para baixo, nível por nível

```
java

// Pré-Ordem (Raiz → Esquerda → Direita)
public void preOrdem(No no) {
    if (no != null) {
        System.out.print(no.valor + " ");
        preOrdem(no.esquerdo);
        preOrdem(no.direito);
    }
}

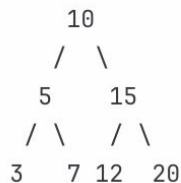
// Em-Ordem (Esquerda → Raiz → Direita)
public void emOrdem(No no) {
    if (no != null) {
        emOrdem(no.esquerdo);
        System.out.print(no.valor + " ");
        emOrdem(no.direito);
    }
}

// Pós-Ordem (Esquerda → Direita → Raiz)
public void posOrdem(No no) {
    if (no != null) {
        posOrdem(no.esquerdo);
        posOrdem(no.direito);
        System.out.print(no.valor + " ");
    }
}
```

Percursos (Traversal)

Usados para processar todos os nós:

- **Pré-ordem:** raiz → esquerda → direita
- **Em-ordem:** esquerda → raiz → direita
- **Pós-ordem** esquerda → direita → raiz
- **Por nível:** do topo para baixo, nível por nível



Pré-ordem: 10, 5, 3, 7, 15, 12, 20

Em-ordem: 3, 5, 7, 10, 12, 15, 20 (ordenado!)

Pós-ordem: 3, 7, 5, 12, 20, 15, 10

1. Bancos de Dados

- Índices de busca são implementados usando **árvores balanceadas**, como **B-trees** e **B+-trees** (derivadas de árvores binárias).
- Usadas para:
 - localizar registros rapidamente
 - otimizar **SELECT**, **WHERE**, **ORDER BY**
 - acessar milhões de linhas com baixa latência

✚ *MySQL, PostgreSQL, Oracle e MongoDB usam estruturas baseadas em árvores.*

2. Sistemas de Arquivos

- Muitos sistemas de arquivos usam árvores para indexar pastas e arquivos:
 - **NTFS (Windows)** → B-tree
 - **ext4 (Linux)** → HTree
- Permite localizar arquivos em milissegundos mesmo em discos enormes.

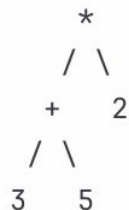
3. Motores de Busca (Search Engines)

- Estruturas em árvore ajudam a:
 - indexar documentos
 - organizar termos
 - acelerar buscas por palavras-chave

Como representar uma expressão Matemática

○

Expressão: $(3 + 5) * 2$



Pós-ordem = $3\ 5\ +\ 2\ *$ (notação polonesa reversa!)

Como representar uma decisão de SIM e NÃO

Nó interno = condição

Ramos = SIM / NÃO

mathematica



Usa-se também um **nó de decisão** (losango), com setas rotuladas:

mathematica



Como representar uma decisão de SIM e NÃO

```
if (cliente.vip = "SIM") {  
    aplicarDescontos(cliente,20);  
} else {  
    aplicarDescontos(cliente,5);  
}
```

Comparações com Lista Encadeadas

Situação	Lista Encadeada	Árvore Balanceada
Acesso rápido a qualquer elemento	✗ Não	✓ Sim ($\log n$)
Ordem dos elementos importa	● Opcional	✓ Mantém ordenação
Inserção/remoção frequente	✓ Muito eficiente	✓ Eficiente ($\log n$)
Busca de valores	✗ Ruim (n)	✓ Boa ($\log n$)
Estrutura hierárquica	✗ Não serve	✓ Sim
Implementação	Simple	Complexa

Algoritmos de busca

1. Busca em Árvores Binárias de Busca (BST – Binary Search Tree)

Esquerda < Raiz < Direita

Explicação

- Compara o valor buscado com o nó atual.
- Decide descer pela esquerda (se for menor) ou direita (se for maior).
- Recursivo e eficiente.

Pseudocódigo

text

```
BuscaBST(no, valor):  
    se no == nulo:  
        retorne "não encontrado"  
  
    se valor == no.valor:  
        retorne no  
  
    se valor < no.valor:  
        retorne BuscaBST(no.esquerda, valor)  
    senão:  
        retorne BuscaBST(no.direita, valor)
```

2. Busca em Árvores Binárias Comuns (não ordenadas)

- Essas árvores **não têm ordenação**, então não dá para seguir esquerda/direita por comparação.
- Existem dois algoritmos clássicos:

2.1. Busca em profundidade (DFS – Depth-First Search)

Explora o nó o mais profundo possível antes de voltar.

3 variações:

- **Pré-ordem (pre-order)**: raiz → esquerda → direita
- **Em-ordem (in-order)**: esquerda → raiz → direita
- **Pós-ordem (post-order)**: esquerda → direita → raiz

Pseudocódigo DFS – Pré-ordem

text

```
DFS_PreOrdem(no, valor):  
    se no == nulo: retorne  
  
    se no.valor == valor:  
        retorne no  
  
    DFS_PreOrdem(no.esquerda, valor)  
    DFS_PreOrdem(no.direita, valor)
```

Algoritmos de busca

2. Busca em Árvores Binárias Comuns (não ordenadas)

- Essas árvores **não têm ordenação**, então não dá para seguir esquerda/direita por comparação.
- Existem dois algoritmos clássicos:

2.2. Busca em largura (BFS – Breadth-First Search)

Explora **nível por nível** usando uma fila.

Pseudocódigo BFS

text

```
BFS(no_raiz, valor):  
    criar fila  
    enfileirar(no_raiz)  
  
    enquanto fila não vazia:  
        no = desenfileirar()  
  
        se no.valor == valor:  
            retorne no  
  
        se no.esquerda != nulo: enfileirar(no.esquerda)  
        se no.direita != nulo: enfileirar(no.direita)
```

Exercícios